

Module 16: Kubernetes for Backend and DevOps Engineers

Questions

Q1. What is Kubernetes, and why is it used?

Answer: Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It provides a declarative API for defining desired application state and continuously works to maintain that state. Kubernetes is used because it abstracts away infrastructure complexity, enables self-healing through pod rescheduling, supports rolling updates and rollbacks, and provides service discovery, load balancing, storage orchestration, and secrets management. For backend developers, it standardizes how applications run across environments. For DevOps engineers, it provides the control plane for running production workloads at scale.

Q2. What is a Kubernetes cluster, and what are its main components?

Answer: A Kubernetes cluster consists of a control plane and worker nodes. The control plane includes the API server, etcd, scheduler, controller manager, and optionally the cloud controller manager. The API server is the entry point for all commands and state changes. etcd is the distributed key-value store for cluster state. The scheduler assigns pods to nodes based on resources and constraints. The controller manager runs controllers that regulate the state of Deployments, ReplicaSets, Services, and other objects. Worker nodes run the kubelet, kube-proxy, and container runtime. The kubelet ensures containers run as specified in pod definitions, while kube-proxy handles network routing and load balancing for Services.

Q3. What is a Pod in Kubernetes?

Answer: A Pod is the smallest deployable unit in Kubernetes. It represents one or more containers that share the same network namespace, IP address, storage volumes, and lifecycle. Containers within a Pod can communicate via localhost. Pods are ephemeral by design; their IP addresses change when they are recreated. Higher-level controllers such as Deployments, ReplicaSets, StatefulSets, and DaemonSets

manage Pods to provide scaling, self-healing, and ordered lifecycle semantics. A backend developer typically does not create Pods directly but instead defines Deployments that create them.

Q4. What is the difference between a Pod and a container?

Answer: A container is a single process runtime instance with its own isolated filesystem and resources. A Pod is a Kubernetes abstraction that wraps one or more containers and provides shared networking and storage. A Pod gives Kubernetes a higher-level unit to schedule, manage, and scale. Containers inside a Pod share the same network interface and can mount the same volumes. A Pod also carries metadata, labels, and lifecycle policies that a bare container does not. In practice, most application Pods contain one main container and sometimes sidecar containers for logging, proxying, or configuration.

Q5. What is a ReplicaSet?

Answer: A ReplicaSet is a Kubernetes controller that ensures a specified number of identical Pod replicas are running at any time. It uses a label selector to identify which Pods it manages and creates or deletes Pods to match the desired replica count. ReplicaSets provide self-healing by replacing failed Pods and horizontal scaling by adjusting replica count. However, users usually work with Deployments rather than ReplicaSets directly because Deployments manage ReplicaSets and provide rolling update and rollback capabilities.

Q6. What is a Deployment in Kubernetes?

Answer: A Deployment is a higher-level controller that manages a ReplicaSet and provides declarative updates for Pods. It is used for stateless applications. A Deployment allows you to define the desired state, including the container image, replica count, strategy, and update parameters. It supports rolling updates that replace old Pods gradually, and it keeps a revision history so you can roll back to a previous version. Deployments are the most common workload object for backend microservices because they handle availability during updates automatically.

Q7. What is the difference between a Deployment and a StatefulSet?

Answer: A Deployment manages stateless Pods that are interchangeable. It supports arbitrary scaling and does not guarantee pod identity or ordering. A StatefulSet is designed for stateful applications that require stable network identity, stable persistent storage, and ordered deployment, scaling, and termination. StatefulSet Pods have predictable names with ordinal indexes such as `web-0` , `web-1` , and `web-2` . Each Pod can have its own PersistentVolumeClaim via a volumeClaimTemplate.

Deployments are ideal for REST services, while StatefulSets are used for databases, message queues, and distributed stateful systems.

Q8. What is a DaemonSet?

Answer: A DaemonSet ensures that one Pod runs on each selected node in the cluster. It is used for node-level infrastructure services such as log collectors, monitoring agents, node exporters, and network proxies. When nodes are added, DaemonSet pods are automatically scheduled on them. When nodes are removed, the associated Pods are garbage collected. Unlike Deployments, DaemonSets are not scaled by replica count because their scale is determined by the number of nodes.

Q9. What is a Kubernetes Namespace?

Answer: A Namespace is a logical partition within a Kubernetes cluster that isolates groups of resources. Namespaces allow multiple teams or applications to share a cluster without name collisions. They are often used to separate environments such as development, staging, and production, or to isolate microservices by team. Resource quotas, limits, RBAC policies, and network policies can be applied per namespace. Objects such as Pods, Services, ConfigMaps, and Secrets exist within a namespace, but cluster-wide resources such as Nodes and PersistentVolumes are not namespaced.

Q10. What is a Kubernetes Service, and why is it needed?

Answer: A Service exposes a set of Pods as a stable network endpoint. Because Pod IPs are ephemeral, clients cannot rely on individual Pod addresses. A Service provides a stable IP, DNS name, and load balancing across matching Pods. Services use label selectors to determine which Pods are members. The main service types are ClusterIP for internal access, NodePort for exposing on a static port on each node, LoadBalancer for cloud provider load balancers, and ExternalName for mapping to an external DNS name. Services are essential for microservices communication within a cluster.

Q11. How does a Kubernetes Service route traffic to Pods?

Answer: A Service selects Pods using label selectors and maintains a list of endpoints through the Endpoints or EndpointSlice API. kube-proxy runs on every node and watches Service and EndpointSlice objects. It configures network rules, typically using iptables or IPVS, so that traffic to the Service IP is translated and load-balanced across the backend Pod IPs. ClusterDNS provides DNS records such as `my-service.my-namespace.svc.cluster.local` so clients can resolve Services by name.

Q12. What is a headless Service?

Answer: A headless Service is a Service with `clusterIP: None`. It does not allocate a virtual IP or perform load balancing. Instead, DNS lookups return the individual Pod IPs directly. Headless Services are used when clients need to discover and communicate with specific Pods, such as in StatefulSets where each Pod has a stable hostname. For example, a headless Service for a StatefulSet allows DNS records like `web-0.my-service.default.svc.cluster.local` to resolve to the specific Pod.

Q13. What are the different Service types?

Answer: The four main Service types are ClusterIP, NodePort, LoadBalancer, and ExternalName. ClusterIP is the default and exposes the Service only within the cluster. NodePort exposes the Service on a static port of every node, making it reachable from outside the cluster through the node's IP. LoadBalancer provisions an external load balancer from a cloud provider and forwards traffic to NodePorts or directly to Pods. ExternalName maps the Service DNS name to an external DNS name and is used for integrating external services into cluster DNS.

Q14. What is a PersistentVolume and a PersistentVolumeClaim?

Answer: A PersistentVolume is a cluster-level storage resource representing a piece of network-attached or cloud-backed storage. A PersistentVolumeClaim is a request for storage by a Pod or workload. Pods use PVCs to mount storage without knowing the underlying storage provider. Kubernetes binds a PVC to a compatible PV based on access mode, storage class, and capacity. With dynamic provisioning, a StorageClass automatically creates a PV when a PVC is submitted. PVCs are namespaced, while PVs are cluster-scoped.

Q15. What are StorageClasses and dynamic provisioning?

Answer: A StorageClass defines the type of storage and the provisioner that creates it. Examples include AWS EBS, GCP Persistent Disk, Azure Disk, and NFS provisioners. When a PVC references a StorageClass, the provisioner automatically creates a matching PV and binds it to the PVC. Dynamic provisioning removes the need for administrators to manually create volumes. It also allows reclaim policies such as Delete or Retain to be defined, determining what happens to the PV when the PVC is deleted.

Q16. What are the access modes for PersistentVolumes?

Answer: Persistent Volumes support three access modes. ReadWriteOnce means the volume can be mounted as read-write by a single node. ReadOnlyMany means many nodes can mount the volume read-only. ReadWriteMany means many nodes can mount the volume read-write simultaneously. The supported access modes depend on the underlying storage technology. Block storage such as EBS supports RWO, while shared filesystems such as NFS or EFS support RWX. Choosing the correct access mode is important for workloads that run on multiple nodes.

Q17. What is Minikube, and when is it used?

Answer: Minikube is a tool that creates a single-node Kubernetes cluster on a local machine for development and learning. It runs the control plane and worker components on the same node, either as a virtual machine, a container, or directly on the host. Minikube is used when developers need a local Kubernetes environment to test manifests, deploy applications, and learn Kubernetes concepts without needing a cloud cluster. It supports addons for ingress, DNS, storage, and dashboard. For production workloads, a managed or self-hosted multi-node cluster is required.

Q18. What is a ConfigMap, and how is it used?

Answer: A ConfigMap is a Kubernetes object for storing non-sensitive configuration data as key-value pairs. It decouples configuration from container images, allowing the same image to be used across environments. ConfigMaps can be mounted as files in a volume, exposed as environment variables, or read by an application through the Kubernetes API. For backend developers, ConfigMaps are useful for application settings such as feature flags, URLs, and logging levels. Sensitive data should not be stored in ConfigMaps; use Secrets instead.

Q19. What is a Secret in Kubernetes?

Answer: A Secret is a Kubernetes object for storing sensitive data such as passwords, tokens, and certificates. Secret values are base64-encoded but are not encrypted by default unless etcd encryption at rest is enabled. Secrets can be mounted as files or injected as environment variables. Best practice is to mount them as files so multiple values can be updated without recreating the Pod, and so secrets do not appear in process listings. For production, integration with external secret managers such as Vault or cloud secret stores is recommended.

Q20. What is a liveness probe, a readiness probe, and a startup probe?

Answer: A liveness probe determines whether the container is running. If it fails, Kubernetes restarts the container. A readiness probe determines whether the container is ready to accept traffic. If it fails,

the Pod is removed from Service endpoints. A startup probe is used for slow-starting containers; while it is running, liveness and readiness checks are disabled. Once the startup probe succeeds, the other probes begin. Probes can be HTTP GET, TCP socket, gRPC, or exec commands. Proper probe configuration is critical for availability and graceful rollouts.

Q21. What is a rolling update in Kubernetes?

Answer: A rolling update is a strategy for replacing old Pods with new ones gradually without downtime. A Deployment creates a new ReplicaSet for the updated version and scales it up while scaling down the old ReplicaSet. The parameters `maxSurge` and `maxUnavailable` control how many extra Pods can exist and how many old Pods can be unavailable during the update. Kubernetes waits for readiness probes on new Pods before terminating old Pods, ensuring the Service always has healthy backends. Rolling updates are the default strategy for Deployments and are essential for zero-downtime deployments.

Q22. How do you roll back a Deployment in Kubernetes?

Answer: Kubernetes keeps a revision history for Deployments. To roll back, you run `kubectl rollout undo deployment/<name>` to revert to the previous revision. You can also specify a revision number with `--to-revision`. The Deployment then scales down the current Pods and scales up the Pods from the selected revision. Rollbacks are useful when a new release introduces errors or performance problems. It is important to set a reasonable `revisionHistoryLimit` so old ReplicaSets are retained for rollback.

Q23. What is a Horizontal Pod Autoscaler?

Answer: The Horizontal Pod Autoscaler automatically adjusts the number of Pod replicas in a Deployment, ReplicaSet, or StatefulSet based on observed metrics such as CPU utilization, memory utilization, or custom metrics. It queries the Metrics API and compares current usage to target thresholds. If usage is above the target, it scales up. If usage is below the target for a sustained period, it scales down. HPA improves resource efficiency and handles traffic spikes automatically. It requires the Metrics Server or a custom metrics adapter to be installed in the cluster.

Q24. What is RBAC in Kubernetes?

Answer: Role-Based Access Control in Kubernetes restricts access to cluster resources based on the roles of users, groups, or service accounts. RBAC uses Roles and ClusterRoles to define permissions,

and RoleBindings and ClusterRoleBindings to assign those permissions to subjects. A Role applies within a namespace, while a ClusterRole applies cluster-wide. RBAC is essential for securing multi-tenant clusters and ensuring that applications and users have only the permissions they need. For example, a backend service account should only be allowed to read ConfigMaps in its own namespace, not delete nodes.

Scenario based questions

S1. A Pod is stuck in `CrashLoopBackOff`. What is your troubleshooting workflow?

Answer: I start with `kubectl get pods -n <namespace>` to confirm the status and restart count. Then I run `kubectl describe pod <pod-name>` to inspect events, which often reveal the cause such as OOMKilled, liveness probe failure, or image pull errors. I read logs with `kubectl logs <pod-name>` for the current container and `kubectl logs --previous` for the last crashed instance. If logs are not enough, I use `kubectl exec -it <pod-name> -- /bin/sh` to inspect the running state, or temporarily add a debug init container. Common causes include missing configuration, invalid startup command, dependency not ready, or insufficient resources. Real-time example: a Spring Boot Pod restarts continuously with `CrashLoopBackOff`. `kubectl describe` shows `Back-off restarting failed container`. `kubectl logs --previous` reveals `Caused by: java.lang.IllegalArgumentException: Could not resolve placeholder 'DB_HOST'`. The ConfigMap `app-config` was not mounted because of a typo in `configMapRef`. I correct the ConfigMap name, apply the manifest, and the Pod starts.

S2. A Pod is in `ImagePullBackOff`. How do you resolve it?

Answer: `ImagePullBackOff` means Kubernetes cannot pull the container image. I check `kubectl describe pod` for the exact error, which may say `manifest unknown`, `unauthorized`, or `pull access denied`. I verify the image name and tag with `docker pull <image:tag>` from a local machine. If the registry is private, I create a `docker-registry` Secret and reference it in the Pod spec via `imagePullSecrets`, or attach it to the ServiceAccount. If the image tag is mutable such as `latest`, I switch to an immutable tag such as a Git commit SHA. I also check network connectivity and registry quotas. Real-time example: a CI pipeline pushes to a private registry but the production Pod fails with `ImagePullBackOff`. The Pod spec references `imagePullSecrets: regcred`, but the secret is in a different namespace. I create the same secret in the application

namespace with `kubectl create secret docker-registry -n prod regcred ...`, and the image pulls successfully.

S3. A Pod is OOMKilled. What do you do?

Answer: I confirm the OOM status with `kubectl describe pod` or `kubectl get pod -o yaml`. I look at memory usage with `kubectl top pod` or Prometheus metrics. If the application legitimately needs more memory, I increase the `resources.limits.memory` and the corresponding JVM or runtime heap setting. If memory usage grows continuously, I investigate for a memory leak using heap dumps, profiling, or application metrics. I always set both requests and limits, with limits higher than requests but not excessively high. Real-time example: a Java service is OOMKilled during batch processing. The container limit is 512 MiB but the JVM uses `-Xmx512m`, leaving no room for native memory. I change the JVM argument to `-Xmx384m` and raise the container limit to 768 MiB. The batch completes without restarts.

S4. A Service is not reachable even though the Pod is running. How do you debug it?

Answer: I check the Service selector against Pod labels with `kubectl get svc` and `kubectl get pods --show-labels`. I verify that the Service has endpoints with `kubectl get endpoints <service-name>`. If endpoints are empty, the selector does not match or the Pods are not ready. I check that the Service `targetPort` matches the container `containerPort`. I test DNS resolution with `nslookup <service-name>.<namespace>.svc.cluster.local` from another Pod. I also inspect NetworkPolicies that may block traffic, and Ingress or load balancer configuration if the Service is exposed externally. Real-time example: a frontend service cannot reach the backend. `kubectl get endpoints backend` returns no endpoints. The backend Pod label is `app: backend-api` but the Service selector is `app: backend`. I correct the selector, the endpoints populate, and the frontend resolves the backend DNS.

S5. A Deployment rolling update is stuck and old Pods are not terminating. Why?

Answer: This usually happens because new Pods are not becoming ready, so Kubernetes cannot terminate old Pods without violating `maxUnavailable`. I check new Pod status and logs with `kubectl get pods` and `kubectl logs`. I look for readiness probe failures, application startup errors, or missing dependencies. If the readiness probe is misconfigured with too short an

`initialDelaySeconds` , new Pods are marked not ready before the application finishes starting. I tune the probe or add a startup probe for slow-starting applications. I also check if `maxUnavailable` is set to zero and `maxSurge` is zero, which prevents any replacement. Real-time example: a Spring Boot service update hangs because the readiness probe hits `/actuator/health` immediately, but the context takes 45 seconds to start. I add a startup probe with `initialDelaySeconds: 10` and `periodSeconds: 10` over 10 attempts, and the rolling update completes.

S6. A StatefulSet Pod loses its data after rescheduling to another node. Why?

Answer: This happens when the StatefulSet does not use a PersistentVolumeClaim template or when the PVC is not bound to a storage class with the correct reclaim policy. In a StatefulSet, each Pod must have its own stable PVC created by `volumeClaimTemplates` . If the Pod uses an emptyDir volume, data is lost when the Pod is deleted or moved. I verify the StatefulSet manifest has `volumeClaimTemplates` , check that the PVCs exist and are bound, and ensure the StorageClass supports the required access mode. Real-time example: a MongoDB replica set uses `emptyDir` for data in a StatefulSet. When a node is drained, the Pod reschedules and restarts with empty data, causing replica set inconsistency. I switch to a `volumeClaimTemplate` backed by a `gp3` StorageClass with `ReadWriteOnce` , and rescheduling preserves the data.

S7. You need to run a log collector on every node. Which workload object do you use?

Answer: I use a DaemonSet. A DaemonSet ensures one Pod runs on each selected node, and automatically adds Pods when nodes join the cluster. Log collectors, monitoring agents like Prometheus node-exporter, and CNI plugins are common DaemonSet workloads. I create the DaemonSet with a toleration if it needs to run on control plane or tainted nodes, and mount the host log directory as a volume. For backend observability, this ensures all application logs are collected regardless of where Pods run. Real-time example: the platform team wants Fluent Bit to ship logs from every node to Elasticsearch. I deploy Fluent Bit as a DaemonSet with a hostPath volume for `/var/log/containers` , tolerations for master nodes, and a resource limit to prevent it from starving workloads. Every node now ships container logs.

S8. A Pod needs to wait for a database to be ready before starting. How do you handle it?

Answer: I use an init container that checks database readiness and exits successfully only when the database accepts connections. For example, an init container running a Postgres client can run `pg_isready -h db -U user` in a loop. Init containers run sequentially before the main container starts. Alternatively, the application can implement retry logic with exponential backoff at startup. The cleanest Kubernetes-native approach is init containers, while application retries provide resilience outside Kubernetes. Real-time example: a Spring Boot application starts before Postgres is ready and crashes repeatedly. I add an init container with `image: postgres:15` and command `['sh', '-c', 'until pg_isready -h postgres -U app; do sleep 2; done']`. The main container starts only after Postgres accepts connections, eliminating `CrashLoopBackOff`.

S9. A Namespace needs resource limits so one team cannot consume all cluster capacity. How do you enforce it?

Answer: I create a ResourceQuota in the namespace that caps CPU, memory, number of Pods, Services, PersistentVolumeClaims, and other objects. I also create LimitRange objects to set default resource requests and limits for containers that do not specify them. ResourceQuota prevents one namespace from starving others, while LimitRange ensures predictable resource profiles. I apply these policies at namespace creation time and document them for developers. Real-time example: the staging namespace runs a load test that consumes all CPU, causing production alerts. I add a ResourceQuota limiting staging to 40 CPU cores, 80 GiB memory, and 50 Pods. I also add a LimitRange with default memory request 256 Mi and limit 512 Mi. The staging load test is throttled, and production stability is restored.

S10. A backend developer wants to expose a service on a custom domain with HTTPS. What Kubernetes resources are involved?

Answer: I use an Ingress resource to route HTTP and HTTPS traffic from the custom domain to a ClusterIP Service. The Ingress references a TLS Secret that contains the certificate and private key. An Ingress controller such as NGINX, Traefik, or AWS Load Balancer Controller must be installed in the cluster to satisfy Ingress resources. Optionally, cert-manager can automatically provision and renew certificates from Let's Encrypt. The Service must select the backend Pods, and the Pods must respond to health checks. For backend developers, the key is that the Service is ClusterIP while Ingress handles external access and TLS termination. Real-time example: a Spring Boot API needs to be available at `api.shop.com` with HTTPS. I create a ClusterIP Service, an Ingress with `host: api.shop.com`, `tls` referencing a Secret `api-shop-tls`, and install NGINX Ingress Controller with a public IP. cert-manager provisions the certificate, and traffic reaches the backend Pods.

S11. A Pod cannot mount a PersistentVolumeClaim. What do you check?

Answer: I check the PVC status with `kubectl get pvc`. If it is Pending, there may be no matching PV or the StorageClass provisioner failed. I read the events with `kubectl describe pvc` and `kubectl describe pod`. Common causes include no available PV for static provisioning, the StorageClass does not exist, the access mode is unsupported by the storage backend, or the requested storage exceeds available capacity. For dynamic provisioning, I check that the CSI driver or cloud provider integration is working and that credentials are correct. Real-time example: a Postgres Pod stays in `ContainerCreating`. `kubectl describe pod` shows `Unable to attach or mount volumes: unmounted volumes=[data] and persistentvolumeclaim postgres-data not found`. The PVC name in the Pod spec was `postgres-data-claim` but the actual PVC was `postgres-data`. I correct the volume name and the Pod mounts successfully.

S12. A Kubernetes secret value needs to be updated. Will running Pods pick it up automatically?

Answer: No, Pods do not automatically pick up Secret changes when the Secret is mounted as environment variables. Environment variables are set at Pod creation time. If the Secret is mounted as a volume, Kubernetes updates the mounted files asynchronously, usually within a few minutes, but the application may not reload them unless it watches the file. For immediate updates, I restart the Pods using `kubectl rollout restart deployment/<name>`. For dynamic secret rotation, I integrate with an external secret operator or Vault agent that injects updated secrets and signals the application. Real-time example: a database password is rotated. The Secret is updated, but the application still uses the old password from environment variables. I perform a rolling restart of the Deployment, and the new Pods start with the updated Secret value.

S13. A multi-tenant cluster needs network isolation between namespaces. How do you implement it?

Answer: I create NetworkPolicy resources with a default-deny rule for ingress and egress in each namespace, then add explicit allow rules for required traffic. Default-deny prevents all Pods in the namespace from receiving or initiating traffic unless allowed. I allow ingress from specific namespaces or Pods using label selectors, and egress to DNS, dependencies, and external endpoints as needed. This ensures teams cannot accidentally access each other's services. Calico, Cilium, and other CNI plugins support NetworkPolicy enforcement. Real-time example: two teams share a cluster. I apply a default-deny ingress NetworkPolicy to each namespace and allow only the frontend namespace to reach the backend API on port 8080. Cross-namespace traffic from unrelated workloads is blocked, reducing the blast radius of a compromised service.

S14. A Deployment needs zero-downtime updates with a minimum number of Pods available. How do you configure it?

Answer: I configure the Deployment strategy with `RollingUpdate`, set `maxUnavailable` to zero or a small number, and set `maxSurge` to allow extra Pods during the update. I also define a `PodDisruptionBudget` to ensure a minimum number of replicas remain available during voluntary disruptions such as node drains and updates. Readiness probes must be accurate so Kubernetes only routes traffic to Pods that are truly ready. For example, `maxUnavailable: 0` and `maxSurge: 1` ensures old Pods stay running until a new Pod is ready. Real-time example: a payment service runs with three replicas. I set `maxUnavailable: 0`, `maxSurge: 1`, and a `PodDisruptionBudget` with `minAvailable: 2`. During a rollout, at least two Pods are always ready, and traffic is never served to a Pod before its readiness probe succeeds.

S15. A backend application needs different configurations per environment. How do you manage it in Kubernetes?

Answer: I use ConfigMaps and Secrets for environment-specific configuration, keeping the container image identical across environments. Each namespace represents an environment such as dev, staging, or prod. I mount ConfigMaps as files or inject values as environment variables. For complex configuration, I use Helm or Kustomize to generate manifests per environment. Sensitive values go into Secrets, and non-sensitive values go into ConfigMaps. I avoid embedding environment differences in the image so that the same artifact can be promoted through the pipeline. Real-time example: the order service needs different database URLs for staging and production. I create a ConfigMap `order-service-config` in each namespace with the appropriate `DB_URL`. The Deployment references the ConfigMap as environment variables. The same Docker image is deployed to both environments, and only the ConfigMap differs.

S16. A Pod is stuck in `Pending` and never schedules. What do you check?

Answer: I run `kubectl describe pod` to read the events, which usually explain why scheduling failed. Common causes include insufficient CPU or memory on nodes, node selectors that match no nodes, taints preventing scheduling, missing PersistentVolume for a PVC, or anti-affinity rules that cannot be satisfied. I check node resources with `kubectl describe nodes` and `kubectl top nodes`. If the issue is resources, I scale the cluster or reduce the Pod's resource requests. If it is scheduling constraints, I review node labels, taints, tolerations, and affinity rules. Real-time example: a

new service requests `cpu: 4` and `memory: 16Gi`, but the cluster nodes only have 2 CPU and 8 GiB each. The Pod stays Pending. I reduce the request to `cpu: 500m` and `memory: 1Gi` based on actual profiling, and the Pod schedules immediately.

S17. A Pod logs sensitive information. How do you prevent it from being collected?

Answer: I update the application to mask or remove sensitive fields before logging. I configure the logging framework to redact known patterns such as tokens, passwords, and credit card numbers. I also restrict access to log storage and ensure log retention policies comply with security requirements. If sensitive data is already in logs, I rotate and purge old logs and investigate how it was introduced. From a DevOps perspective, I avoid mounting logs to host paths where multiple users can read them, and I use centralized logging with role-based access. Real-time example: an API logs request headers including `Authorization: Bearer <token>`. I update the logging interceptor to replace the `Authorization` value with `[REDACTED]` before writing to stdout. Existing logs in Elasticsearch are purged according to retention policy, and access to log indices is restricted to the security team.

S18. A Kubernetes cluster needs to scale Pods based on custom business metrics such as queue depth. How do you set it up?

Answer: I use the Horizontal Pod Autoscaler with custom metrics. The application exposes a custom metric through a metrics endpoint or through Prometheus Adapter. I configure the HPA to scale on that metric instead of CPU. For example, an order processing service can scale based on the number of messages in a Kafka queue. The Prometheus Adapter translates Prometheus queries into the `custom.metrics.k8s.io` API. The HPA then adjusts replicas to keep the metric near a target value. I also set min and max replicas to control scaling bounds. Real-time example: an email notification service is backed by a Redis queue. I expose queue length as a Prometheus metric, configure Prometheus Adapter to serve it as `custom.metrics.k8s.io|queues|email_queue_length`, and create an HPA targeting 100 messages per replica with `minReplicas: 2` and `maxReplicas: 20`. When a marketing campaign floods the queue, the service scales automatically.

S19. A StatefulSet requires ordered scaling and stable storage. How do you design the manifest?

Answer: I use a StatefulSet with a `serviceName` referencing a headless Service, `volumeClaimTemplates` for stable storage, and a `podManagementPolicy` of `OrderedReady` for ordered startup and shutdown. Each Pod gets a stable hostname like `kafka-`

`0.kafka.default.svc.cluster.local`. The `volumeClaimTemplate` creates one PVC per Pod with a stable name, so data persists across rescheduling. I set an update strategy of `RollingUpdate` with an appropriate partition if I need controlled rollout. For backend services that require identity such as database replicas or Kafka brokers, this design is essential. Real-time example: a Kafka cluster runs as a `StatefulSet` with three replicas. Each broker has a headless Service `kafka`, a `StatefulSet` `kafka`, and a `volumeClaimTemplate` using the `fast-ssd` StorageClass. Broker 0 always has hostname `kafka-0.kafka` and mounts the same PV, even after node failure. This stability is required for Kafka broker identity and persistent log storage.

S20. A junior backend developer asks why they cannot directly create a Pod in production. How do you explain?

Answer: I explain that a bare Pod is not self-healing, not scalable, and not rolling-updatable. If the node fails or the Pod crashes, Kubernetes will not recreate it automatically. A Deployment provides replicas, health monitoring, rolling updates, and rollback. A Pod also has an ephemeral IP that changes on restart, while a Service provides stable networking. In production, developers should define Deployments or StatefulSets, not raw Pods. I also mention that organizational policy may enforce this through admission controllers or RBAC. Real-time example: a developer creates a standalone Pod for a batch job. The node is drained for maintenance, the Pod is evicted, and the job never restarts. I show them how to use a Job or CronJob for batch work, and a Deployment for long-running services, so Kubernetes manages lifecycle and recovery.